Seize codes pour quinze valeurs : un codon synonyme sans perte dans MXFP4, et gpt-oss-120b servi depuis le disque sur un portable de 16 Go

Stone

29 octobre 2026

Résumé

MXFP4, le format sur 4 bits où gpt-oss publie les poids de ses experts, a seize codes pour quinze valeurs : +0 et −0 sont deux codes du même zéro. Le quantificateur de gpt-oss garde le signe des valeurs arrondies à zéro ; sur les 19 110 297 600 poids d'experts de gpt-oss-20b, ce signe est une pièce juste (entropie 0,9999999967 bit), et aucun calcul des moteurs considérés n'en dépend. Nous démontrons que fondre −0 dans +0 avant le codage entropique fait gagner exactement $\mathbb{P}(V = 0)\, H(S \mid V = 0)$ bit par poids, deux fois la part des codes −0 quand le signe est une pièce juste, et nous le vérifions sur les 180 tenseurs d'experts de gpt-oss-20b et gpt-oss-120b à $1{,}7 \times 10^{-4}$ bit près. Porté dans la version 1.2 du format `.trc` du moteur troncons par ses seules tables de codage, décodeur et noyaux inchangés, le fondu retire 1 800 486 912 octets (1,80 Go) à gpt-oss-120b, 0,12562 bit par poids d'expert (3,26 % de ses bits), et les logits du moteur restent identiques au bit sur 5 120 pas, sur les deux modèles. troncons sert gpt-oss-120b (63,4 Go) sur un portable à 16 Go de RAM et RTX 4060 de 8 Go, avec 384 Mo de travail sur l'hôte et une empreinte sur la carte fixée par elle, non par le modèle (environ 6,3 Go de ses 8 Go). Sur ce portable, en séries ABBA, le fichier v1.2 génère 2,890 jetons/s contre 1,988 pour le v1.0 (×1,454 ; un juge indépendant : ×1,442), là où ses 3,3 % d'octets en moins permettraient au plus +3,4 % : une expérience croisée sur gpt-oss-20b, à l'étage disque, montre que le gain suit le motif des requêtes de lecture que le format impose à ce disque. C'est un résultat de cette machine, non transportable. Nous rapportons aussi ce qui n'a pas marché, chacun contre son seuil déclaré, et la méthode : seuils commités avant de mesurer, témoins qui peuvent tomber, juges indépendants.

1 Introduction

Le projet dont vient cet article s'est donné un but concret : faire tourner, sur un portable ordinaire, un modèle de langage bien plus gros que toute la mémoire du portable, *exactement* (au bit, les logits que le même moteur calcule sur les poids non compressés), en stockant ses poids compressés sur le disque et en ne décodant que ce dont un calcul a besoin, à l'instant où il en a besoin. La machine a 16 Go de RAM, une carte RTX 4060 Laptop de 8 Go et un disque NVMe de 1 To chiffré par BitLocker. Le modèle est gpt-oss-120b [1], un mélange d'experts de 63,4 Go dans son format natif MXFP4 : quatre fois la RAM, huit fois la mémoire de la carte.

Trois tomes techniques rapportent le travail en entier, chacun en anglais et en français. Le Tome I [2] a mesuré la machine et montré que, sur elle, tout se ramène au nombre de bits lus sur le disque par poids. Le Tome II [3] a construit le moteur, troncons, et jugé son premier jalon (J1) contre douze critères déclarés avant la première ligne de code. Le Tome III [4] a cherché une manière de lire moins de bits et construit ce qui a tenu. Cet article court est leur porte d'entrée. Il rapporte une idée petite et exacte, ce qu'elle a fait sur cette machine, et il dit clairement ce qui n'a pas marché.

L'idée est un synonyme. Dans le code génétique, plusieurs codons donnent le même acide aminé [5] ; dans MXFP4, deux codes donnent le même zéro. Quand le signe d'un zéro ne porte aucune information, ce qui est le cas dans gpt-oss, le stocker coûte presque un bit par zéro, et environ un poids d'expert sur huit est un zéro. Le retirer est sans perte tant que chaque lecteur des codes lit les deux codes comme zéro, et c'est une propriété qui se démontre sur le vrai noyau. Le code de trop du zéro est connu, et d'autres formats lui

donnent une autre valeur (section 2) ; nous ne le faisons pas, parce que nous gardons le modèle tel qu'il a été publié, chaque poids avec la même valeur, et cessons seulement de payer un signe qui ne change aucune valeur.

Contributions.

- Une mesure, tenseur par tenseur sur les 180 tenseurs d'experts de deux modèles publiés, que le signe d'un zéro est une pièce juste, et l'énoncé exact, élémentaire, de ce que fait gagner le fondu du synonyme, $H(C) = H(V) + \mathbb{P}(V = 0)\, H(S \mid V = 0)$ (sections 3.2 et 4.1).
- Une démonstration que le noyau MXFP4 du produit matrice–vecteur du moteur rend les mêmes bits sur les deux codes, et un lemme qui montre que le fondu se porte dans un format par ses tables de codage, sans toucher au décodeur (sections 3.2 et 3.3).
- Les fichiers `.trc` v1.2 de gpt-oss-20b et de gpt-oss-120b, vérifiés expert par expert en valeur, 1,80 Go plus petits sur le 120b, avec des logits identiques au bit (sections 4.2 et 4.3).
- Le jalon J1 refait sur gpt-oss-120b en séries ABBA, et l'attribution de son ×1,45 au motif de lecture du format sur ce disque, par une expérience croisée d'un juge indépendant sur gpt-oss-20b, à l'étage disque (sections 4.4 et 4.5).
- Des résultats négatifs avec leurs seuils déclarés, et la discipline de mesure qui a produit tout ce qui précède (sections 3.4 et 5).

Les étiquettes suivent les tomes : [mes.] mesuré sur cette machine, outil et journal commités ; [calc.] calculé sur des mesures commitées ; [litt.] littérature, non reproduite. Les chemins comme `genome/06-codon-v1-2.md` sont relatifs au dépôt troncons (section 7).

2 Travaux liés

Servir des modèles plus gros que la mémoire. FlexGen répartit les poids et le cache entre carte, processeur et disque pour maximiser le débit d'une génération par lots avec une seule carte [6]. LLM in a Flash lit les poids sur une mémoire flash à la demande, avec un fenêtrage et un groupement lignes–colonnes qui allongent les lectures [7]. PowerInfer sépare les neurones en un ensemble chaud gardé sur la carte et un ensemble froid calculé sur le processeur [8]. Pour les mélanges d'experts, Eliseev et Mazur [9] gardent un cache LRU d'experts sur la carte et en chargent d'autres par anticipation ; MoE-Infinity conçoit un cache d'experts pour les machines personnelles [10] ; Fiddler calcule les experts froids sur le processeur au lieu de les déplacer [11]. llama.cpp répartit les couches MoE entre l'hôte et la carte et sert de base au Tome II [12]. troncons diffère par son contrat : les poids restent *codés entropiquement sans perte* sur le disque et sont décodés sur la carte au moment de servir, et les logits sont, au bit, ceux que le moteur calcule sur les poids bruts du GGUF ; le cache d'experts et sa théorie (l'optimalité de Belady [13], les bornes de LRU) sont au Tome II.

Compression sans perte des poids. ZipNN [14], DFloat11 [15] et Huff-LLM [16] compressent les poids sans perte, surtout en codant entropiquement la part redondante des représentations flottantes, comme les exposants du BF16. La compression avec perte sous le bit de très grands MoE existe aussi [17]. Nous travaillons un niveau plus bas, dans un format qui est déjà sur quatre bits : MXFP4 [18, 19] range des blocs de 32 codes E2M1 qui partagent une échelle E8M0. La redondance que nous exploitons n'est pas un biais statistique mais un *synonyme* : deux codes, une valeur, par le zéro signé d'IEEE 754 [20]. Cette redondance est connue. BitMoD remplace le zéro redondant de variantes FP3 et FP4 par une valeur spéciale [21] ; le recyclage de code de NxFP réaffecte le code −0 des formats signe-grandeur à la moitié du plus petit niveau de quantification, et l'applique à MXFP4 [22] ; RaZeR réaffecte le zéro redondant de NVFP4 à des valeurs de quantification de plus [23]. Ce sont trois formats neufs, avec perte, dans lesquels un modèle se quantifie à nouveau, et aucun ne mesure la fréquence des −0 dans un modèle publié. Le codage sans perte a été étendu au FP8 et au FP4 [24], mais, pour le FP4, il ne compresse que les échelles des blocs et range les valeurs sur 4 bits sans les compresser, leurs flux lui paraissant statistiquement uniformes ; dans gpt-oss-20b, l'entropie des codes vaut 3,8638 bits par poids, et non 4 (section 3.2), et notre codeur les conditionne par la classe d'échelle.

Nous ne connaissons pas de travail qui mesure, dans un modèle déjà publié en MXFP4, si le signe d'un zéro porte une information, et qui le retire sans perte, avec une démonstration que le noyau rend les mêmes bits. Le codeur entropique est rANS entrelacé [25, 26] ; les outils de théorie de l'information sont classiques [27] ; le juge de la qualité est la divergence de Kullback–Leibler [28] contre une référence en double précision.

Des idées essayées qui ont échoué. Le routeur d'une couche MoE découpe l'espace de ses entrées en régions polyédriques, comme le fait un polynôme tropical [29] ; les matrices de poids des réseaux entraînés montrent souvent des spectres à queue lourde [30], qu'on éprouve contre une loi de puissance par la méthode de Clauset *et al.* [31]. La section 5 rapporte ce que ces lectures ont donné sur gpt-oss.

3 Méthode

3.1 La machine, le modèle et le moteur

La machine [mes.]. Windows 11, un i7-13620H (16 fils), 16 Go de RAM (15,6 utilisables), une carte RTX 4060 Laptop de 8 188 Mio sur un lien PCIe 4.0 ×8, un disque NVMe Samsung de 1 To derrière Intel RST VMD, sous BitLocker. Le Tome I a mesuré 2,82 Go/s depuis le disque (`fondations/journaux/01-machine/debits-mesures.json`).

Le modèle [litt.] [mes.]. gpt-oss-120b a 36 couches de 128 experts, dont le routeur de chaque couche en choisit quatre par jeton ; son fichier GGUF fait 63 387 346 208 octets, ses 4 608 experts 61,07 Go, et chaque jeton lit en plus 1,69 Go d'attention, de routeurs et de tête de sortie (`mesures/gpt-oss.md`). Son jumeau de développement, gpt-oss-20b (24 couches de 32 experts, 12 109 566 624 octets), est déjà plus gros que la carte.

Le moteur et le chemin des octets. troncons est un espace de travail Rust dont les noyaux CUDA sont chargés par la seule API du pilote (Tome II, chapitres 1 à 6). Un fichier `.trc` par modèle, aligné sur 4 096 octets, porte un *tronçon* par expert : les trois matrices MXFP4 de l'expert, codées entropiquement par rANS entrelacé sur 32 voies, chaque voie sachant par sa position quelle table lire (une table par classe d'échelle κ), plus un répertoire qui donne le décalage, la longueur et l'empreinte de chaque tronçon. Les poids résidents (attention, routeurs, tête) vivent en VRAM ; les experts vivent dans un cache en VRAM à emplacements fixes. Quand un routeur demande un expert absent des emplacements (un défaut), son tronçon traverse quatre étages :

1. **du disque à l'hôte** : une lecture sans tampon (`FILE_FLAG_NO_BUFFERING`) par tronçon, confiée au système sans attendre et récoltée sur un port de complétion [32], directement dans une mémoire hôte épinglée ;
2. **de l'hôte à la carte** : une copie DMA depuis cette même page épinglée, par le PCIe ; la carte n'a qu'un moteur de copie sous WDDM, un fait que le Tome II a mesuré ;
3. **le décodage** : un décodeur rANS en CUDA fait du tronçon un emplacement d'expert, et il est démontré qu'il rend exactement le décodage scalaire ;
4. **le calcul** : un noyau MXFP4 du produit matrice–vecteur (GEMV) lit l'emplacement ; chaque réduction suit un arbre fixe, si bien que deux exécutions sur les mêmes octets rendent les mêmes bits.

Une garde de production compare l'empreinte de chaque tronçon lu au répertoire et fait de toute corruption une erreur fatale nommée. Les résultats du Tome II sur gpt-oss-120b, au format v1.0 : un ensemble de travail de pointe sur l'hôte de 384 Mo (384,19 Mo, mémoire épinglée comprise, contre 380,50 Mo pour le 20b, cinq fois plus petit), là où llama.cpp tient 10,93 Go de mémoire hôte dans sa meilleure configuration publiée, tandis que la mémoire engagée privée de troncons, 6,93 Go (llama.cpp : 6,81 Go), est, selon la lecture du Tome II (qui la publie sans la juger), de l'ordre de ses allocations sur la carte et de sa mémoire épinglée, réservée contre la limite d'engagement de Windows sans occuper la RAM physique ; les deux moteurs se servent aussi de la carte, troncons d'environ 6,3 Go de ses 8 Go (6 317 303 600 octets), une part fixée par la carte et non par le modèle, et nvidia-smi lit la carte entière, autres processus compris, à une pointe de 6 574 Mio sous troncons et de 6 026 et 6 027 Mio sous llama.cpp dans cette configuration (épreuve indépendante du Tome II,

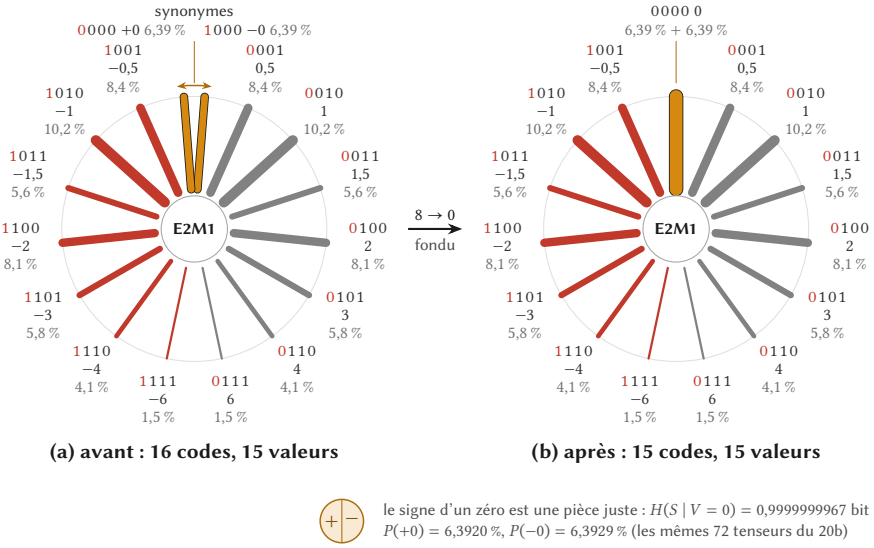


Figure 1 – Le fondu $8 \to 0$ retire l'un des deux codes du zéro. Les quinze valeurs d'E2M1 à pas égal sur un cercle (0 en haut ; valeurs de la norme OCP, que llama.cpp double). L'épaisseur d'un rayon croît avec la part mesurée de son code dans les 72 tenseurs d'experts de gpt-oss-20b (19 110 297 600 poids), de façon affine (0,4 pt + 44 pt × part) : le rayon du zéro s'épaissit donc de ×1,88, et non de ×2. Avant : `0000` (+0, 6,3920 %) et `1000` (−0, 6,3929 %) pour une valeur ; après, un seul rayon porte les deux parts. Le signe d'un zéro est une pièce juste. Analogie, non preuve : GGU, GGC, GGA et GGG donnent tous la glycine. [mes.] Données : `mesures/genome/predictif-elagage/histo-toutes-gptoss20b.npz`, `mesures/genome/etat-de-l-art/journaux/zero-compte-20b.json`.

`mesures/moteur/epreuve-J1/csv/s1-2026-10-16-passes.csv`, qui porte aussi la mémoire engagée), si bien que le grand écart entre les deux moteurs est dans l'ensemble de travail de l'hôte ; des logits identiques au bit entre le fichier codé et le fichier brut sur 5 120 pas, par le même moteur ; 1,965 jeton/s avec la garde, ×2,22 la meilleure configuration de llama.cpp mesurée (0,884 jeton/s ; deux séances du même matin, Tome II) ; et, sur la fenêtre 0 (255 positions) avec un cache KV en f16, une divergence KL de 0,00308 contre une référence f64, contre 0,0607 pour llama.cpp [mes.] (Tome II, chapitre 7 ; `mesures/moteur/O/`, `mesures/moteur/epreuve-J1/`). Le Tome II attribue l'essentiel de l'écart de llama.cpp, sur le 20b, à son arrondi de la rampe YaRN [33] plutôt qu'à la précision des noyaux ; avec un cache KV en f32, la KL de troncons vaut −0,00048, au plancher de résolution des fichiers de logits eux-mêmes (`.k1d`, $\log q$ sur 16 bits). La mémoire de l'hôte ne suit plus la taille du modèle.

3.2 Le codon synonyme du zéro

E2M1 a un bit de signe et trois bits de grandeur. llama.cpp et troncons doublent ses grandeurs en entiers, $K = (0, 1, 2, 3, 4, 6, 8, 12)$, et passent le facteur 2 dans l'échelle. Pour un code $c \in \{0, \dots, 15\}$ de bits $c_3 c_2 c_1 c_0$,

$$v(c) = (-1)^{c_3}\, K[c \,\&\, 7]. \tag{1}$$

L'application v est injective sauf sur $\{0, 8\}$: $v(0) = +0$ et $v(8) = -0$, et $-0 = +0$ en IEEE 754. Seize codes, quinze valeurs (figure 1).

La pièce [mes.]. Le quantificateur de gpt-oss garde le signe d'une valeur arrondie à zéro. Sur les 72 tenseurs d'experts de gpt-oss-20b, le code +0 a la part 6,3920 % et −0 la part 6,3929 % : le signe d'un zéro a une entropie de 0,9999999967 bit. L'entropie des codes vaut 3,8638 bits par poids, celle des valeurs 3,7360 ; la différence, 0,1278 bit, est ce que coûte un signe qui ne change aucune valeur (`mesures/genome/etat-de-l-art/journaux/zero-compte-20b.json`).

Proposition 1 (Entropie d'un code à synonyme)**.** *Soit C le code d'un poids, $V = v(C)$ sa valeur et $S = C_3$ son bit de signe. Alors $H(C) = H(V) + \mathbb{P}(V = 0)\, H(S \mid V = 0)$.*

Démonstration. V est une fonction de C, donc $H(C, V) = H(C)$, et par la règle de chaîne $H(C, V) = H(V) + H(C \mid V)$. Sachant $V = v \neq 0$, C est déterminé, puisque v est injective hors de 0. Sachant $V = 0$, $C \in \{0, 8\}$ est déterminé par S. D'où $H(C \mid V) = \mathbb{P}(V = 0)\, H(S \mid V = 0)$. □

Corollaire 2 (Le gain du fondu)**.** *Remplacer C par $C' = \gamma(C)$, avec $\gamma(8) = 0$ et γ l'identité ailleurs, met C' en bijection avec V : l'entropie baisse de $\mathbb{P}(V = 0)\, H(S \mid V = 0)$ bit par poids. Si le signe d'un zéro est une pièce juste, les codes -0 sont la moitié des zéros et*

$$\textit{gain} = \mathbb{P}(V = 0) = 2 \times (\textit{part des codes } -0). \tag{2}$$

Il en va de même conditionnellement à tout contexte X connu du décodeur (classe d'échelle, octet voisin) : remplacer partout $H(\cdot)$ par $H(\cdot \mid X)$. Si le contexte prédisait le signe, le gain serait inférieur à $\mathbb{P}(V = 0)$.

Remarque 3 (Les octets). Le codeur travaille sur des octets, deux codes à la fois. Le fondu retire de l'alphabet les octets dont un demi-octet vaut 8 : $16 + 16 - 1 = 31$ octets, il en reste donc $256 - 31 = 225$ [calc.]. Appliquée à l'octet et à sa paire de valeurs, la même démonstration donne $\mathbb{P}(V = 0)$ bit par poids quand les signes des zéros sont des pièces justes, indépendantes de tout le reste.

Le fondu n'est sans perte que si *chaque* lecteur des codes donne au code 8 la valeur 0. Le projet les a relevés (`genome/06-codon-v1-2.md` § 1) : ggml lit les codes par une table int8 dont l'entrée 8 vaut 0 [litt.] ; le décodeur de la carte recopie des octets sans lire de valeurs ; la référence f64 lit le GGUF, pas le `.trc` ; et le seul lecteur de valeurs dans les noyaux du moteur est la GEMV, où le code 8 vaut −0,0f. La question est donc celle des *bits* d'une somme dont un terme passe de -0 à $+0$.

Proposition 4 (La GEMV rend les mêmes bits ; Tome III, proposition A1)**.** *La GEMV MXFP4 du moteur (`moteur/crates/trc-noyaux/noyaux/gemv.cuh`, dans sa version du commit `4473050`, celle de l'arbre au commit `000cef9`) part de $s = +0$ pour chaque accumulateur et avance par $s \leftarrow \mathrm{fma}(v, x, s)$ dans un ordre fixe, v étant la grandeur doublée et signée du code. Si les entrées x sont finies, remplacer le code 8 par le code 0 ne change aucun bit de s, à aucun pas.*

Esquisse. (0) Tout binary32 fini est un multiple de 2^{-149} et v est entier : toute somme exacte $vx + s$ est sur la grille $2^{-149}\mathbb{Z}$; si elle n'est pas nulle, son arrondi ne l'est pas. (1) L'accumulateur ne vaut jamais -0 : il part de $+0$; une somme exacte nulle s'arrondit à $+0$ sauf si ses deux termes valent -0, ce que s n'est pas (par récurrence) ; une somme non nulle ne s'arrondit pas à zéro, par (0). (2) Pour $v = \pm 0$, $vx = \pm 0$ exactement ; si $s \neq 0$, $s' = s$; si $s = 0$, alors $s = +0$ et $s' = +0$ quel que soit le signe de vx. (3) Les autres pas ont des opérandes égaux au bit. Par récurrence, s est égal au bit dans les deux calculs ; la réduction en arbre fixe additionne ensuite des sommes partielles égales au bit, et l'échelle et le biais s'appliquent après. □

La démonstration utilise le fait que les grandeurs sont doublées en entiers et que l'échelle reste hors de l'accumulation ; un noyau qui ferait entrer l'échelle, ou une GEMM qui partirait de -0, en sortirait. Le projet a donc écrit un invariant dans son architecture : tout lecteur de codes lit le code 8 comme zéro, et tout noyau neuf qui lit des codes porte un témoin qui compare un expert à sa copie canonisée. La même récurrence vaut pour la référence f64 sous une hypothèse, dite au Tome III : aucune somme exacte non nulle ne s'arrondit à zéro par dépassement inférieur.

3.3 Le format `.trc` v1.2

Le codeur canonise les codes de chaque expert ($8 \to 0$) avant de compter les symboles et de construire ses tables rANS : les 31 octets qui portent un demi-octet 8 reçoivent la fréquence zéro. Rien d'autre ne change (tableau 1).

Lemme 5 (L'invariant des tables suffit)**.** *Si la table d'un contexte donne $f[s] = 0$ à un symbole s, aucun flux, juste ou corrompu, ne fait produire s au décodeur.*

Démonstration. Le décodeur lit la fente $i = x \bmod 4096$ de son état x et rend le symbole qui la porte ; par construction, la fente i porte le s tel que $c[s] \leq i < c[s] + f[s]$, c les fréquences cumulées. Si $f[s] = 0$, l'intervalle est vide et aucune fente ne porte s, quel que soit x. □

Table 1 – Ce qui change de `.trc` v1.0 à v1.2, et rien d'autre.

Où	Décalage	v1.0	v1.2
en-tête, version mineure	10 (u16)	0	**2**
en-tête, drapeaux	12 (u32)	bits 0 à 2	+ **bit 3, canonique**
répertoire, chaque entrée	56 (u64)	réservé, 0	empreinte de l'expert canonique
tables CODES_CTX	—	256 fréquences	construites sur les codes canoniques ; fréquence 0 pour les 31 octets qui portent un 8, vérifiée à l'ouverture

Source : `decisions/ARCHITECTURE.md` § 4.16 ; `genome/06-codon-v1-2.md` § 2. Le lecteur refuse le bit 3 sous une version mineure inférieure à 2, sur un fichier brut ou à deux niveaux, une empreinte canonique sans le bit 3, et une table canonique qui donne une fente à un octet portant un 8 ; chaque refus a son essai et sa cassure.

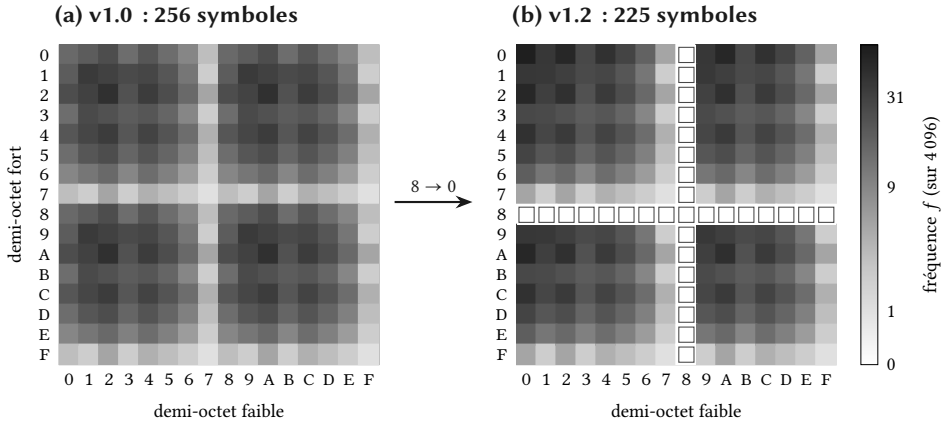


Figure 2 – Après le fondu, la ligne et la colonne 8 s'éteignent : il reste 225 octets sur 256. La table CODES_CTX ($\kappa = 1$) de gpt-oss-120b, couche 18, matrice up, en grille 16×16 (demi-octet fort en ligne, faible en colonne) ; niveau de gris $\log_{10}(1 + f)$, f la fréquence sur 4 096. Cases blanches bordées : fréquence nulle, absentes. Leur masse passe sur la ligne et la colonne 0. Sur l'ensemble des 540 tables CODES_CTX des deux modèles, un second lecteur écrit depuis la seule spécification trouve une fente pour un octet portant un 8 dans 540 tables v1.0 (13 % des fentes en moyenne) et dans aucune table v1.2. [mes.] Données : `mesures/genome/construction/codon/tables-octets.csv`, `tables-octets.json`.

Deux conséquences d'ingénierie en découlent. La garantie « aucun flux v1.2 ne produit de −0 » se contrôle une fois, sur les tables, à l'ouverture ($36 \times 3 \times 3 \times 31 = 10\,044$ lectures pour le 120b [calc.]), et non à chaque symbole. Et *un flux v1.2 est un flux v1.0 dont l'alphabet effectif a 225 octets* : le décodeur de la carte et la GEMV n'ont pas une ligne de changée. Un synonyme entre dans un format par ses tables, pas par son décodeur. Comme un fichier v1.2 n'est plus égal au bit au GGUF mais égal en valeur, chaque entrée du répertoire garde deux empreintes, celle d'origine (la provenance) et la canonique (ce que le décodeur doit rendre), et le vérificateur compare des valeurs, pas seulement des octets.

3.4 Comment les mesures ont été faites

Les nombres de cet article ont été produits sous des règles que le projet applique à chaque étape, et qui comptent autant que les nombres.

- **Seuils déclarés avant de mesurer.** Chaque critère et son seuil sont écrits et commités avant la première passe ; le seuil dérive d'une grandeur mesurée et nommée. Les critères de l'étape v1.2 (A-C1 logits, A-C2 taille, A-C3 vitesse) sont dans l'en-tête de `outils/genome/construction/codon/pilote_codon.py`, commit `a50c380`, avant toute passe ; l'amendement de la séance calme s2 est le commit `71905dd`, avant ses passes.
- **Des témoins qui peuvent tomber, et des cassures.** Chaque mesure a un témoin dont la réponse est connue d'avance, et chaque témoin neuf est cassé exprès pour vérifier que l'instrument tombe. Exemples : le codeur lancé sans l'option neuve doit écrire la v1.0 octet pour octet (même SHA-256 que le fichier du

Table 2 – Ce que le fondu rend sur le disque [mes.].

	gpt-oss-20b	gpt-oss-120b
poids d'experts	19 110 297 600	114 661 785 600
bits de flux par poids, v1.0 → v1.2	3,85057 → 3,72266	3,85456 → 3,72892
octets de tronçons, v1.0 → v1.2	9 226 493 952 → 8 920 932 352	55 416 086 528 → 53 615 599 616
gain par poids d'expert (A-C2 : ≥ 0,12)	**0,12791 bit**	**0,12562 bit**
octets en moins	305 561 600	1 800 486 912
fichier `.trc`, v1.0 → v1.2	11 157 405 696 → 10 851 844 096	57 730 662 400 → 55 930 175 488
plus grand tronçon, v1.0 → v1.2	12 144 640 → 11 755 520	12 251 136 → 11 878 400

Source : `mesures/genome/construction/codon/tailles.json`, `tailles-par-tenseur.csv` (outil `outils/genome/construction/codon/analyse_codon.py`). La ligne des bits de flux compte les flux codés des poids d'experts (médianes, δ et codes ; leur différence, depuis les valeurs non arrondies, vaut 0,12563 bit sur le 120b) ; la ligne du gain compte les octets de tronçons. Le lecteur du format écrit par le juge redonne les quatre tailles de fichier à l'octet (`genome/JUGEMENT-construction.md` § 2).

15 octobre) ; une médiane faussée dans un expert doit faire différer les logits (512 lignes sur 512) ; un demi-octet 0 changé en 8 dans un expert décodé doit faire tomber les contrôles d'octets et laisser tenir le contrôle de valeur.

- **Séries ABBA.** Un coût ne se fonde jamais sur une passe : des processus neufs alternent A B B A A B B A, et le bruit est l'écart d'un programme à lui-même. Un gain n'est annoncé que si $r - 1 > 2\sigma$, σ l'écart-type relatif des passes A ; on déclare l'absence de régression si $r \geq 1 - 2\sigma$. Les passes tournent sur secteur, sous un verrou sur la carte, et sont écartées si un autre processus chargeait la machine (plus de 5 % d'un cœur ou 8 Mo lus en 3 s).
- **Des juges indépendants.** Chaque étape est refaite par un juge qui n'a écrit ni la décision ni le code, avec ses propres outils : ici, un décodeur du format écrit depuis sa seule description, un lecteur GGUF, des fenêtres de texte neuves que personne n'avait vues, une séance de vitesse à trois fichiers, et un lecteur de disque à lui.
- **Les phrases barrées.** Quand une mesure contredit une phrase publiée, même la nôtre, la phrase est barrée là où elle est, et ce qui a changé s'écrit à côté. Cet article porte une telle correction (section 4.5).

4 Résultats

4.1 Le gain, tenseur par tenseur

Le codeur a fondu 1 221 698 587 demi-octets 8 sur gpt-oss-20b (6,393 % de ses poids d'experts) et 7 199 491 948 sur gpt-oss-120b (6,279 %) ; le vérificateur, qui canonise les tranches du GGUF par son propre chemin, compte les mêmes nombres à l'unité [mes.] (`genome/06-codon-v1-2.md` § 5). Le corollaire 2 prédit donc $2 \times 0{,}0639288 = 0{,}12786$ bit (20b) et $2 \times 0{,}0627889 = 0{,}12558$ bit (120b) ; le gain mesuré sur les tronçons codés vaut 0,12791 et 0,12562 bit [calc.]. Sur chacun des 180 tenseurs d'experts (72 du 20b, 108 du 120b), l'écart entre le gain mesuré et deux fois la part des codes −0 reste entre −0,00017 et +0,00016 bit, +0,00005 en moyenne (figure 3). C'est plus fort qu'il n'y paraît : si la classe d'échelle ou l'octet voisin, que le codeur connaît tous deux, prédisaient le signe d'un zéro, le gain mesuré tomberait sous $\mathbb{P}(V = 0)$. Dans chaque tenseur, le signe d'un zéro est une pièce juste que ne prédisent ni la classe d'échelle ni l'autre demi-octet de son octet, et le codeur rANS en récupère le bit entier. Le gain va de 0,115 bit (120b, couche 26, up) à 0,239 bit (120b, couche 0, up, où 11,9 % des poids sont des −0).

4.2 Les tailles sur le disque

Le gain par poids d'expert est la différence des octets de tronçons, en bits, divisée par le nombre de poids d'experts ; sur le 120b [calc.],

$$\frac{(55\,416\,086\,528 - 53\,615\,599\,616) \times 8}{114\,661\,785\,600} = 0{,}12562 \text{ bit},$$

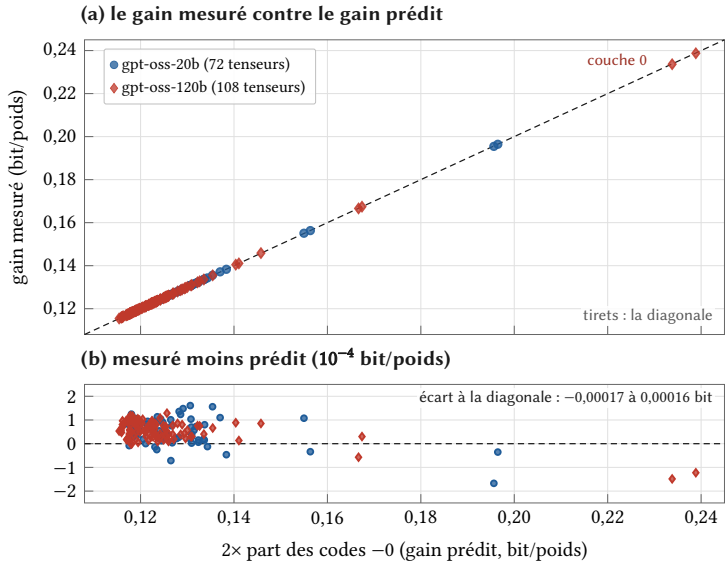


Figure 3 – Les 180 tenseurs d'experts des deux modèles tombent sur la diagonale que prédit le corollaire 2. (a) Gain mesuré du fondu (bits par poids d'expert, flux de codes de chaque tenseur) contre deux fois la part des codes −0 ; les deux points de la couche 0 du 120b sont les plus riches en zéros. (b) Mesuré moins prédit, en 10^{-4} bit par poids : le contexte κ et l'alphabet octet n'écartent aucun tenseur au-delà de −0,00017 à +0,00016 bit. [mes.] [calc.] Données : `mesures/genome/construction/codon/tailles-par-tenseur.csv`.

soit 0,12562/3,85456 = 3,26 % des bits par poids d'expert, et 3,25 % des octets de tronçons (tableau 2). Le seuil A-C2, déclaré à 0,12 bit, tient sur les deux modèles. Si le disque seul limitait la vitesse, ces octets permettraient 1/0,96751 − 1 = 3,4 % de jetons par seconde en plus sur le 120b [calc.].

4.3 L'exactitude

Les experts, en valeur [mes.]. Le vérificateur décode chaque expert des deux fichiers v1.2 et contrôle huit propriétés, dont les octets décodés contre le GGUF canonisé par un second chemin, l'absence de tout 8 et l'égalité *en valeur* de chaque poids : 768 experts sur 768 et 4 608 sur 4 608 sont exacts, zéro faute à chaque contrôle. Ses deux cassures trouvent exactement l'expert visé ; la seconde (un 0 décodé changé en 8 : même valeur, octets faux) fait tomber les contrôles d'octets et laisse tenir le contrôle de valeur, ce qui est exactement la distinction qu'introduit la v1.2 (`genome/06-codon-v1-2.md` § 5). Le juge a décodé 96 experts tirés au hasard avec son propre décodeur, écrit depuis la seule description du format : 96 sur 96 égalent l'expert du GGUF au bit près, au signe des zéros près, et les demi-octets qui changent entre les décodages v1.0 et v1.2 sont exactement les codes −0 du GGUF.

La GEMV sur la carte [mes.]. Un expert synthétique aux dimensions de gpt-oss (trois matrices 2880 × 2880), dont le signe est tiré pour toute grandeur, porte 2 486 216 demi-octets 8 ; contre sa copie canonisée, avec des entrées quelconques (mantisses pleines, exposants de 2^{-10} à 2^{10}, zéros des deux signes, sommes non exactes), 8 640 **lignes de sortie, aucune différente au bit** ; une cassure (un code 0 de la copie mis à 1) fait différer la ligne 0.

Les logits [mes.]. `troncons evaluer` calcule l'empreinte sur 64 bits des logits f32 à chacune des 5 120 positions de dix fenêtres de 512 jetons. v1.2 contre v1.0 : 5 120 sur 5 120 égales sur gpt-oss-20b et sur gpt-oss-120b, et les fichiers de KL égaux octet pour octet (`genome/06-codon-v1-2.md` § 7). Sur le 120b, la v1.2 par le préremplissage par flux et le programme du 17 octobre rend au bit les logits que la v1.0 rendait par la boucle de pas et le programme du 15 octobre. Le juge l'a refait sur dix fenêtres que personne n'avait vues : 0 empreinte différente sur 5 120 sur chaque modèle, et une cassure à lui fait différer 512 sur 512 (`genome/JUGEMENT-construction.md` § 4). La proposition 4 tient pour le vrai moteur, sur un texte neuf.

Table 3 – J1 sur gpt-oss-120b, séance s2 : les huit passes, puis les médianes par jeton des quatre passes de chaque fichier [mes.].

passe	1 A	2 B	3 B	4 A	5 A	6 B	7 B	8 A
jetons/s	1,981	2,873	2,892	1,994	2,006	2,897	2,887	1,776

médiane par jeton	A, v1.0	B, v1.2
génération, jetons/s	1,988	2,890
octets de tronçons lus	710 118 798	686 756 916
défauts (taux de succès du cache)	59,08 (0,5897)	59,08 (0,5897)
temps de lecture, s	0,391	0,259
débit de lecture vu par le moteur, Go/s	1,81	2,65
décodage sur la carte, ms	62,1	42,7
copie de l'hôte à la carte, ms	96,2	68,2
ensemble de travail de pointe, Mo	388,67	382,21

Source : `mesures/genome/construction/codon/j1-s2-passes.csv`, `j1-s2-verdicts.json` ; médianes par `memoire/article/outils/medianes_j1.py` (témoin : ses médianes de jetons/s égalent celles du fichier des verdicts).

4.4 Le jalon J1 sur gpt-oss-120b, v1.2 contre v1.0

La tâche du jalon est fixée depuis le Tome I : une invite wikitext, puis 256 jetons gloutons. Huit processus neufs alternent A B B A A B B A, A le fichier v1.0 d'origine (écrit le 15 octobre), B le fichier v1.2 ; le cache est fixé à 326 emplacements, le nombre du J1, pour que le plus grand tronçon plus court de la v1.2 ne libère pas un emplacement et ne mêle pas deux effets. La séance s2 a tourné dans la nuit du 21 au 22 octobre, sur une machine calme ; aucune de ses passes n'a été prise sous charge [mes.].

- **Les jetons** : les 256 identifiants de jetons sont égaux dans les huit passes, et égaux au texte de toutes les passes du 120b des 15 et 16 octobre ; défauts, taux de succès et emplacements sont identiques. Le format ne change ni le routage ni le cache.
- **Les octets**, déterministes : 686 756 916/710 118 798 = 0,9671, comme prédit (0,967).
- **La vitesse** : médianes 1,988 et 2,890 jetons/s, $r = 1{,}454$, $\sigma = 5{,}6\,\%$; rapports appariés 1,450, 1,451, 1,444 et 1,626. Au seuil déclaré : pas de régression, et **gain annoncé** ($0{,}454 > 2\sigma = 0{,}113$).
- **La mémoire** : −6,45 Mo ; 4 472 832 octets en sont les 12 tampons d'arrivée épinglés, dimensionnés sur le plus grand tronçon, $12 \times (12\,251\,136 - 11\,878\,400)$ [calc.] ; environ 2,0 Mo ne sont pas expliqués.
- **La reproduction** : le juge a mis les trois fichiers du 120b dans une seule séance équilibrée (A B R R B A A B R R B A, R un compagnon avec perte), aucune des 36 passes des deux séances du juge (120b et 20b) sous charge : $r(B/A) = 2{,}889/2{,}004 = 1{,}442$, $\sigma = 5{,}8\,\%$ (`genome/JUGEMENT-construction.md` § 6.2).

4.5 D'où vient la vitesse : le motif de lecture, sur ce disque

3,3 % d'octets en moins ne peuvent pas rendre 45 % (figure 4b) ; le temps de lecture par jeton baisse d'un tiers (tableau 3), à presque autant d'octets. Dans la même séance s2, l'étage disque seul (`troncons disque`, sans calcul) lit le v1.0 d'origine à 1,742 et 1,741 Go/s et la v1.2 à 2,254 et 2,391 [mes.]. Un troisième fichier, le v1.0 recodé le 17 octobre, égal au bit à l'original, se lit à 1,356 et 1,331 Go/s : il porte 267 473 extents contre 65, et sa fragmentation lui coûte ×1,26 sur le chemin de lecture du moteur (`genome/JUGEMENT-placement.md` § 5.9).

Des deux fichiers v1.0 de même contenu, l'étape de construction a d'abord conclu que l'écart était une affaire de disposition physique, et son juge l'a suivie :

> ~~Ce gain n'est pas attribuable au format. […] La part du format, elle, est bornée par le rapport des octets lus : au plus +3,4 % à disque limitant.~~

(Texte d'origine, en français.) Cette phrase a été barrée le 27 octobre (`genome/06-codon-v1-2.md` § 8), après qu'un second juge, celui du placement sur le disque, eut lu *un seul et même fichier* avec les frontières de requête de l'autre format, sans rien réécrire (tableau 4, figure 5a). Aux seuils déclarés avant (bruit propre

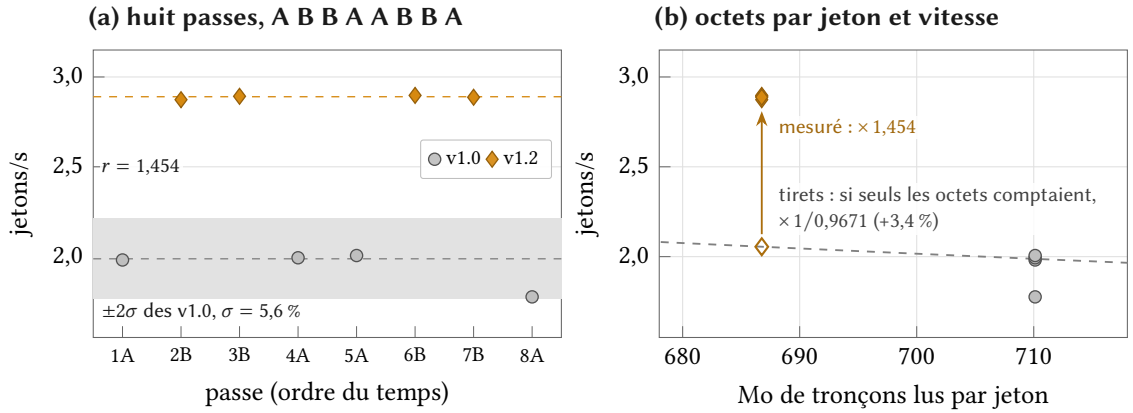


Figure 4 – Sur le 120b, la v1.2 va ×1,454 plus vite que la v1.0, et ses octets ne lui donneraient que +3,4 % au plus. Séance s2, machine calme, huit processus neufs, les mêmes 256 jetons dans tous. (a) Les passes dans l'ordre du temps ; en tirets, les médianes 1,988 (v1.0) et 2,890 (v1.2) jetons/s ; bande grise, $\pm 2\sigma$ des passes v1.0. (b) Les octets lus par jeton sont déterministes (×0,9671) ; si leur quantité seule comptait, la v1.2 serait sur la courbe en tirets, au losange creux. [mes.] Données : `mesures/genome/construction/codon/j1-s2-passes.csv`, `j1-s2-verdicts.json`.

Table 4 – Le motif croisé : gpt-oss-20b, étage disque seul, le même fichier lu avec les frontières de requête de l'autre format, Go/s (juge du placement sur le disque, séance du 26 octobre) [mes.].

Cas	Quatre passes	Médiane
fichier v1.0, motif natif	1,792 ; 1,793 ; 1,813 ; 1,809	1,801
fichier v1.0, motif de la v1.2	2,509 ; 2,493 ; 2,483 ; 2,564	2,501
fichier v1.2, motif natif	2,511 ; 2,487 ; 2,556 ; 2,545	2,528
fichier v1.2, motif de la v1.0	1,811 ; 1,829 ; 1,805 ; 1,774	1,808

Source : `mesures/disque/placement/juge/seance/Jb3-*` ; `genome/JUGEMENT-placement.md` § 5.3.

$\leq 3{,}2\,\%$, $S_j = 11{,}8\,\%$), le fichier v1.0 de gpt-oss-20b lu au motif de son fichier v1.2 va ×1,39 plus vite (1,388 depuis les médianes non arrondies, 1,389 dans le texte du juge depuis les arrondies), et le fichier v1.2 lu au motif v1.0 va ×0,715 : *la lenteur suit le motif, pas le fichier.* Deux expériences de plus bornent la variable. Des requêtes raccourcies de 3,5 % rendent le fichier v1.0 rapide (×1,421) et des requêtes allongées de 3,5 % rendent le fichier v1.2 lent (×0,713) ; mais couper chaque tronçon v1.0 en deux requêtes de 6 Mo, mêmes octets, le ralentit encore (×0,851 à huit lectures en vol, ×0,882 à quatre). La longueur seule n'est pas la variable. En description, les requêtes lentes finissent exactement à une fin de tronçon v1.0 ou débordent de 3,5 % une fin de tronçon v1.2 ; les rapides s'arrêtent 3,5 % avant une fin de tronçon v1.0, ou finissent exactement à une fin de tronçon v1.2 ; ce n'est pas un mécanisme, et la variable exacte n'est pas isolée (`genome/JUGEMENT-placement.md` §§ 5.3, 5.6, 5.7).

Sur le 120b, la séance à trois fichiers du juge partage le rapport de bout en bout entre un toit pour les octets et un reste [calc.] :

$$r(B/A) = 1{,}442 = \underbrace{1/0{,}9671}_{\text{octets : } \leq 1{,}034} \times \underbrace{1{,}394}_{\text{motif : un reste}} ,$$

l'étage disque seul, dans la même séance, lisant B 1,37 fois plus vite que A (figure 5b). Le reste est une borne basse de l'effet du motif. **Sur ce portable, le format v1.2 rend le moteur environ 45 % plus rapide sur gpt-oss-120b, par le motif de ses requêtes de lecture et non par ses 3,3 % d'octets ; ce gain ne se transporte pas à un autre disque sans y être mesuré.** L'amplification de l'étage disque (×1,37) au moteur (×1,44) reste une hypothèse : la carte, qui attend davantage le disque, tournerait à des horloges plus basses ; ses temps de copie et de décodage par jeton diffèrent bien plus que les octets (tableau 3).

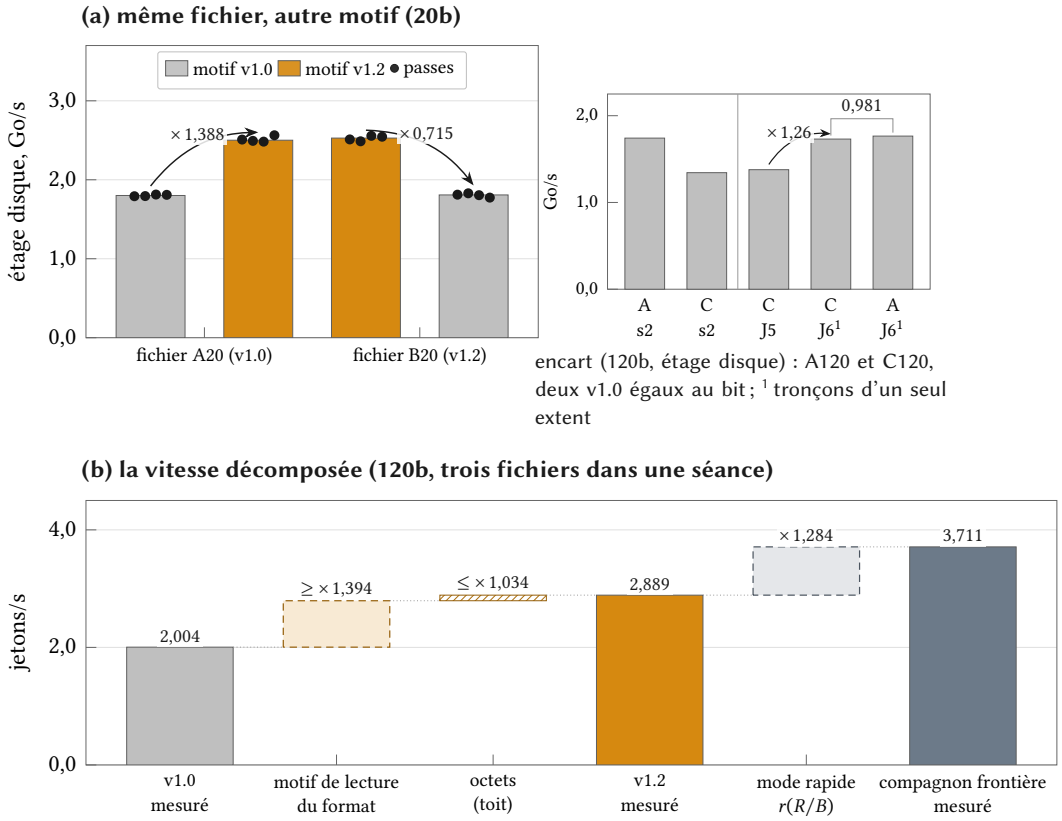


Figure 5 – Le gain vient du motif de lecture qu'impose le format v1.2, sur ce disque. (a) gpt-oss-20b, étage disque seul : le même fichier lu avec les frontières de requête de l'autre format prend la vitesse de ce format (barres : médianes ; points : les quatre passes ; couleur : le motif de lecture ; ×1,388 est calculé depuis les médianes non arrondies). Encart, gpt-oss-120b : A120 et C120 ont le même contenu au bit ; C120 est plus lent à cause de ses 267 473 extents (contre 65) ; sur ses tronçons d'un seul extent, il se lit ×1,26 plus vite que sur des tronçons tirés au hasard, et comme A120 sur les mêmes tronçons (0,981). (b) La séance à trois fichiers du juge sur le 120b : $r(B/A) = 1{,}442$ se partage entre les octets (hachuré, un toit) et le motif (un reste, donc une borne basse) ; le compagnon rapide, avec perte, ajoute ×1,284 contre la v1.2. [mes.] Données : `mesures/disque/placement/juge/seance/`, `mesures/genome/construction/codon/disque-s2.json`, `mesures/genome/construction/juge/figures/`.

5 Ce qui n'a pas marché

Le Tome III a essayé plus qu'il n'a gardé. Chaque essai ci-dessous avait son seuil commité avant la lecture, et chaque verdict a été refait par un juge avec son propre code (tableau 5).

Une garde sur la marge du routeur. Le Tome II avait trouvé que les bascules de routage portent l'essentiel de la divergence entre deux calculs proches : imposer le routage de la référence en retire 88 %. Le routeur d'une couche note 32 experts et en garde quatre ; l'espace de ses entrées est découpé en régions polyédriques, et la marge entre le 4e et le 5e score mesure la distance à une face [29]. L'idée était une garde qui ne refait en exact que les cellules de faible marge. Seuil déclaré : attraper au moins 80 % des bascules en signalant au plus 10 % des cellules. Mesuré sur gpt-oss-20b : 2 905 bascules attrapées sur 5 087 (57,1 %) pour 12 264 cellules signalées sur 122 640 (10 %) ; attraper 80 % demanderait d'en signaler 26,6 %. La garde tombe. Son juge ramène l'échec à deux choses : les couches ne se ressemblent pas (les couches 8 à 23 n'attrapent que 38 à 60 % de leurs bascules), et les bascules secondaires des cascades viennent à des marges encore larges ; la cascade verticale stricte, une bascule en amenant une autre à la couche suivante, n'explique qu'un quart des bascules (`genome/pistes/tropicale/JUGEMENT.md` §§ 4.2, 12).

Une loi de puissance dans les spectres. L'espoir était que les matrices de poids se décrivent par peu de paramètres, comme une fractale. Le test ajuste une loi de puissance aux valeurs singulières de 48 matrices

Table 5 – Résultats négatifs et ouverts, chacun contre son seuil déclaré.

Idée	Seuil déclaré	Mesuré, et verdict
garde sur la marge du routeur	$\geq 80\,\%$ des bascules pour $\leq 10\,\%$ des cellules	57,1 % pour 10 % ; 80 % coûtent 26,6 % : **tombe**
loi de puissance des spectres	≥ 32 matrices sur 48	0 sur 48 (9 au critère amendé) : **ne s'écrit pas**
routage conditionné par le cache	KL contre le moteur sans biais $\leq 0{,}038$ à $-10\,\%$ de défauts	0,0314, borne haute de l'intervalle par fenêtres 0,0384 : **à la limite**
cinq autres pistes de cache	déclaré pour chaque piste (Tome III, chapitre 6)	**ouvertes**, toutes les cinq
mode rapide, génération	$r(R/B) \geq 1{,}769$ (120b) et $\geq 1{,}227$ (20b) : gain annoncé $\times(1-2\sigma)$	×1,284 et ×0,942 contre la v1.2 : **ne tient pas**

Source : `genome/pistes/tropicale/seuil-declare.md`, `JUGEMENT.md` ; `genome/pistes/spectres/seuil.md`, `JUGEMENT.md` ; `genome/mimetisme/P3/resultat-routeur-ligne.md` ; `genome/JUGEMENT-construction.md` § 6 ; Tome III, chapitre 13.

de gpt-oss-20b tirées avant de regarder, par la méthode de Clauset *et al.* [31]. Seuil déclaré : au moins deux tiers de l'échantillon, 32 matrices. Résultat : 0 sur 48 au critère principal, 9 sur 48 « compatibles » à un critère amendé avant la lecture. Le juge a ajouté un théorème : le critère principal ne pouvait jamais être satisfait, à aucune taille, parce que la loi de puissance est la frontière de la famille log-normale tronquée. Ce qui tient, et a été jugé : les 48 matrices ont toutes plus de queue que leur version permutée (48 sur 48). Que cette queue siège là où se trouvent quelques lignes géantes, et qu'une échelle par ligne la capture déjà, vient d'une mesure exploratoire, ni déclarée ni jugée (`genome/pistes/spectres/JUGEMENT.md` ; `genome/pistes/spectres/spectres.md` § 4.3).

Six pistes empruntées à d'autres disciplines. Quarante-cinq candidats venus de la biologie, de la physique, des mathématiques, de l'ingénierie, des systèmes humains et des sciences de la Terre ont été rangés en six pistes pour le cache d'experts (mémoire à deux échelles de temps, étage de RAM exclusif, routage conditionné par le cache, préchargement à confiance calibrée, recuit, fidélité décidée à l'appel). Aucune ne tient nettement sous son juge, chacune contre des seuils déclarés pour elle (Tome III, chapitre 6). Cinq restent ouvertes, chacune pour une raison dite (entre autres des traces trop courtes, un recouvrement non mesuré, un simulateur non validé, des poids synthétiques). Une est à la limite : un routeur qui favorise les experts déjà en VRAM retire environ 10 % des défauts pour une divergence KL de 0,0314 contre le même moteur sans le biais, au bord de la tolérance déclarée. Un résultat de côté est ferme : l'étage de RAM inclusif du moteur n'ajoute rien tant qu'il n'est pas plus grand que le cache en VRAM (`genome/mimetisme/`).

Le mode rapide, avec perte, comme levier de vitesse. Une « mutation synonyme » avec perte des experts du 20b gagne en qualité contre la frontière avec perte (différence appariée $\Delta\mathrm{KL} = -0{,}1565 \pm 0{,}0334$ sur des fenêtres neuves, 4,7 σ). Mais les gains annoncés du mode rapide, ×2,00 sur le 120b et ×1,24 sur le 20b, comparaient des fichiers neufs au vieux fichier v1.0 (sur le 120b, le compagnon rapide était la frontière avec perte : la mutation du 120b n'a jamais été écrite) ; contre la v1.2, que l'étage disque lit à la même vitesse, ils deviennent ×1,284 et ×0,942 [mes.] (`genome/JUGEMENT-construction.md` § 6). Les idées génératives de la ronde (une graine et ses coefficients, des motifs partagés, des experts dupliqués) étaient tombées plus tôt sur les bits : la duplication rend 0,0034 bit par poids, dont 0,0016 par hasard (Tome III, chapitres 3 et 4).

6 Limites : ce qui pourrait être faux

– **Une machine, un disque, des mesures non répliquées ailleurs.** Chaque nombre de vitesse vaut pour ce NVMe, derrière Intel RST VMD, sous BitLocker, des jours donnés. Le ×1,45 de la v1.2 repose sur le motif de lecture sur ce disque ; la variable exacte du motif n'est pas isolée, et son mécanisme (pile de

stockage, contrôleur, données repliées hors du cache SLC) n'est pas observable d'ici. Sur un autre disque le gain peut être nul, ou plus grand ; la seule borne de vitesse transportable est le +3,4 % des octets.

- **L'attribution repose sur le 20b, à l'étage disque.** Le motif croisé a été mesuré sur gpt-oss-20b, étage disque seul ; sur le 120b, la séance à trois fichiers ne donne qu'une borne basse de l'effet du motif. Le motif croisé a été mesuré quatre jours après la séance principale du juge et après une réoptimisation du disque ; le rapport B/A n'a pas bougé (1,398 et 1,362 avant, 1,404 et 1,406 après), ce qui rassure sans prouver.
- **Une famille de modèles, née en MXFP4.** Le gain vaut deux fois la part des codes −0. Un modèle quantifié par un outil qui écrit toujours +0 n'a rien à gagner (la v1.2 lui coûte alors zéro octet) ; un modèle où le signe d'un zéro serait prévisible gagnerait moins. Les 180 tenseurs disent que ni l'un ni l'autre n'est le cas de gpt-oss.
- **La démonstration couvre le noyau d'aujourd'hui.** La proposition 4 suppose des entrées finies et se démontre sur la GEMV du commit `4473050`. Le commit `0be464a`, le noyau d'aujourd'hui, a fait du lecteur des valeurs un paramètre de patron et garde exactement les mêmes `fmaf` en mode exact ; c'est la version qui était dans l'arbre quand le témoin des 8 640 lignes a été commité (`a50c380`). Un noyau futur qui lirait les codes autrement pourrait casser l'égalité au bit. Le corollaire de la référence f64 demande une hypothèse d'absence de dépassement inférieur, mesurée sur cinq vecteurs, non démontrée pour toute entrée.
- **Des emplacements égaux isolent le format, pas le produit.** Avec le nombre d'emplacements par défaut, le plus grand tronçon plus court libère 4 472 832 octets de VRAM, moins d'un emplacement (13 254 144 octets) ; le nombre par défaut ne change probablement pas, mais ce n'est pas mesuré. Environ 2,0 Mo de l'écart de mémoire ne sont pas expliqués.
- **Les échantillons et le texte.** Les résultats négatifs reposent sur des échantillons (12 tenseurs de référence, 48 matrices, 32 ou 768 experts) et sur un seul corpus (wikitext) ; la garde, sur gpt-oss-20b seulement.
- **Les juges partagent des conventions.** Le juge de la construction a écrit son propre décodeur et son propre code de KL, mais sur le même moteur et avec la même convention de KL ; un défaut commun aux deux ne se verrait pas. Un juge s'est trompé une fois (« l'état des fichiers »), et il a été barré ; rien ne garantit qu'aucune autre lecture ne viendra.

7 Conclusion

MXFP4 donne au zéro deux codes. Dans gpt-oss, le choix entre eux est une pièce juste dont aucun calcul des moteurs considérés ne dépend, et son prix est exactement ce que dit le corollaire 2 : 0,12562 bit par poids d'expert sur gpt-oss-120b, 1,80 Go sur le disque. Le fondu entre dans un format par ses seules tables de codage, avec une démonstration que le noyau rend les mêmes bits et une mesure que les logits le font. Sur le portable où tout cela a été mesuré, le nouveau format a aussi rendu le moteur environ 45 % plus rapide, pour une raison qui tient au motif de lecture du format sur ce disque et non à ses octets ; ce nombre est à remesurer avant d'être cru ailleurs. Les autres idées, certaines belles, sont tombées à des seuils déclarés avant d'être lus. Le Tome II dit comment un modèle de 63,4 Go se sert depuis un ensemble de travail de 384 Mo sur l'hôte et une empreinte sur la carte fixée par la carte, ses logits égaux au bit à ceux du fichier brut ; le Tome III dit toute la quête, sa ronde, sa construction et ses pistes.

Disponibilité

Le moteur troncons (espace de travail Rust et noyaux CUDA), les outils, les journaux de mesure commités dont chaque nombre de cet article est tiré, et les trois tomes [2–4], chacun en anglais et en français, sont publiés comme rapport technique à l'adresse `https://github.com/stonearie1491-sudo/troncons`. Les chemins cités dans cet article sont relatifs à la racine de ce dépôt. Les figures se régénèrent depuis les données commitées par des scripts commités (`memoire/outils/tome3_codon.py`, `memoire/article/figures/`).

Références

[1] OpenAI : gpt-oss-120b & gpt-oss-20b model card. arXiv :2508.10925, 2025. URL https://arxiv.org/abs/2508.10925. Consulté le 2026-10-15.

[2] Stone : troncons : faire tourner un modèle bien plus grand que la mémoire. Mémoire de recherche, parties I et II : les fondations mesurées (Tome I). Rapport technique, 2026. Versions anglaise et française ; https://github.com/stonearie1491-sudo/troncons.

[3] Stone : troncons, Tome II : le système. Rapport technique, 2026. Versions anglaise et française ; https://github.com/stonearie1491-sudo/troncons.

[4] Stone : troncons, Tome III : le génome des poids. Rapport technique, 2026. Versions anglaise et française ; https://github.com/stonearie1491-sudo/troncons.

[5] F. H. C. Crick : The origin of the genetic code. *Journal of Molecular Biology*, 38(3):367–379, 1968.

[6] Ying Sheng *et al.* : FlexGen : High-throughput generative inference of large language models with a single GPU. arXiv :2303.06865 (ICML 2023), 2023. URL https://arxiv.org/abs/2303.06865. Consulté le 2026-10-15.

[7] Keivan Alizadeh *et al.* : LLM in a flash : Efficient large language model inference with limited memory. arXiv :2312.11514 (ACL 2024), 2023. URL https://arxiv.org/abs/2312.11514. Consulté le 2026-10-15.

[8] Yixin Song *et al.* : PowerInfer : Fast large language model serving with a consumer-grade GPU. arXiv :2312.12456 (SOSP 2024), 2023. URL https://arxiv.org/abs/2312.12456. Consulté le 2026-10-15.

[9] Artyom Eliseev et Denis Mazur : Fast inference of mixture-of-experts language models with offloading. arXiv :2312.17238, 2023. URL https://arxiv.org/abs/2312.17238. Consulté le 2026-10-15.

[10] Leyang Xue *et al.* : MoE-Infinity : Efficient MoE inference on personal machines with sparsity-aware expert cache. arXiv :2401.14361, 2024. URL https://arxiv.org/abs/2401.14361. Consulté le 2026-10-15.

[11] Keisuke Kamahori *et al.* : Fiddler : CPU-GPU orchestration for fast inference of mixture-of-experts models. arXiv :2402.07033 (ICLR 2025), 2024. URL https://arxiv.org/abs/2402.07033. Consulté le 2026-10-15.

[12] The ggml authors : llama.cpp, version b11515, 2026. URL https://github.com/ggml-org/llama.cpp/releases/tag/b11515. commit 3d65c90d ; MIT ; Consulté le 2026-10-15.

[13] Laszlo A. Belady : A study of replacement algorithms for a virtual-storage computer. *IBM Systems Journal*, 5(2):78–101, 1966.

[14] Moshik Hershcovitch, Andrew Wood, Leshem Choshen, Guy Girmonsky, Roy Leibovitz *et al.* : ZipNN : Lossless compression for AI models. arXiv :2411.05239, 2024. URL https://arxiv.org/abs/2411.05239. Consulté le 2026-10-15.

[15] Tianyi Zhang, Mohsen Hariri, Shaochen Zhong, Vipin Chaudhary, Yang Sui *et al.* : 70% size, 100% accuracy : Lossless LLM compression for efficient GPU inference via dynamic-length float (DFloat11). arXiv :2504.11651 (NeurIPS 2025), 2025. URL https://arxiv.org/abs/2504.11651. Consulté le 2026-10-15.

[16] Patrick Yubeaton, Tareq Mahmoud, Shehab Naga, Pooria Taheri, Tianhua Xia *et al.* : Huff-LLM : End-to-end lossless compression for efficient LLM inference. arXiv :2502.00922, 2025. URL https://arxiv.org/abs/2502.00922. Consulté le 2026-10-15.

[17] Elias Frantar et Dan Alistarh : QMoE : Practical sub-1-bit compression of trillion-parameter models. arXiv :2310.16795, 2023. URL https://arxiv.org/abs/2310.16795. Consulté le 2026-10-16.

[18] Open Compute Project : OCP microscaling formats (MX) specification, version 1.0, 2023. URL https://www.opencompute.org/documents/ocp-microscaling-formats-mx-v1-0-spec-final-pdf. Consulté le 2026-10-15.

[19] Bita Darvish Rouhani, Ritchie Zhao, Ankit More, Mathew Hall *et al.* : Microscaling data formats for deep learning. arXiv :2310.10537, 2023. URL https://arxiv.org/abs/2310.10537. Consulté le 2026-10-16.

[20] IEEE : IEEE standard for floating-point arithmetic (IEEE std 754-2019), 2019.

[21] Yuzong Chen, Ahmed F. AbouElhamayed, Xilai Dai, Yang Wang, Marta Andronic, George A. Constantinides et Mohamed S. Abdelfattah : BitMoD : Bit-serial mixture-of-datatype LLM acceleration. arXiv :2411.11745 (v1, 2024-11-18 ; v2, 2025-04-26 ; HPCA 2025), 2024. URL https://arxiv.org/abs/2411.11745. Consulté le 2026-10-29.

[22] Yun-Chen Lo, Gu-Yeon Wei et David Brooks : Nanoscaling floating-point (NxFP) : NanoMantissa, adaptive microexponents, and code recycling for direct-cast compression of large language models. arXiv :2412.19821 (v1, 2024-12-15), 2024. URL https://arxiv.org/abs/2412.19821. Consulté le 2026-10-29.

[23] Yuzong Chen, Xilai Dai, Jake Hyun, Chi-Chih Chang, Wonsuk Jang, Yuheng Wu, Thierry Tambe, Jae-sun Seo et Mohamed S. Abdelfattah : RaZeR : Pushing the limits of NVFP4 quantization with redundant zero remapping. arXiv :2501.04052 (v1, 2025-01-06 ; v3, 2026-01-31), 2025. URL https://arxiv.org/abs/2501.04052. Consulté le 2026-10-29.

[24] Anat Heilper et Doron Singer : Lossless compression of neural network components : Weights, checkpoints, and K/V caches in low-precision formats. arXiv :2508.19263 (v1, 2025-08-20), 2025. URL https://arxiv.org/abs/2508.19263. Consulté le 2026-10-29.

[25] Jarek Duda : Asymmetric numeral systems : entropy coding combining speed of Huffman coding with compression rate of arithmetic coding. arXiv :1311.2540, 2013. URL https://arxiv.org/abs/1311.2540. Consulté le 2026-10-15.

[26] Fabian Giesen : Interleaved entropy coders. arXiv :1402.3392, 2014. URL https://arxiv.org/abs/1402.3392. Consulté le 2026-10-15.

[27] Thomas M. Cover et Joy A. Thomas : *Elements of Information Theory*. Wiley-Interscience, Hoboken, NJ, 2nd édition, 2006.

[28] S. Kullback et R. A. Leibler : On information and sufficiency. *The Annals of Mathematical Statistics*, 22(1):79–86, 1951.

[29] Liwen Zhang, Gregory Naitzat et Lek-Heng Lim : Tropical geometry of deep neural networks. arXiv :1805.07091, 2018. URL https://arxiv.org/abs/1805.07091. Consulté le 2026-10-17.

[30] Charles H. Martin et Michael W. Mahoney : Implicit self-regularization in deep neural networks : Evidence from random matrix theory and implications for learning. arXiv :1810.01075, 2018. URL https://arxiv.org/abs/1810.01075. Consulté le 2026-10-17.

[31] Aaron Clauset, Cosma Rohilla Shalizi et M. E. J. Newman : Power-law distributions in empirical data. *SIAM Review*, 51 (4):661–703, 2009.

[32] Microsoft : I/O completion ports, 2025. URL https://learn.microsoft.com/en-us/windows/win32/fileio/i-o-completion-ports. Consulté le 2026-10-15.

[33] Bowen Peng, Jeffrey Quesnelle, Honglu Fan et Enrico Shippole : YaRN : Efficient context window extension of large language models. arXiv :2309.00071, 2023. URL https://arxiv.org/abs/2309.00071. Consulté le 2026-10-15.